Cassava 2 — Environment Setup

Oct 26, 2026 · Anas Alsirafy

Cassava models were trained on a local RTX 5090 workstation and on Kaggle notebooks, then timed on a Jetson Xavier NX and a Raspberry Pi 4. Everything is Python with TensorFlow/Keras.

Hardware

Machine	Used for	Notes
Workstation, NVIDIA RTX 5090	Main training	Runs under WSL2 on Windows. Ensemble inference: 89 ms/image.
Kaggle notebook (GPU model to confirm — written as P100)	Training when the workstation was busy	Sessions time out; a 384×384 fine-tuning fold takes hours.
NVIDIA Jetson Xavier NX	Inference timing only	Ensemble: 65 ms/image. Pi Camera v2 would not start on either CSI port, so timing used pre-captured images.
Raspberry Pi 4	Inference timing only	Ensemble: 254 ms/image.

Software

Package	Version	Used for
Python	3.x (to confirm)	Everything
TensorFlow / Keras	2.19 (to confirm)	Models, training, `tf.data` pipelines
albumentations	to confirm	RandomResizedCrop, flips, transpose
opencv-python	to confirm	Reading images, CLAHE
scikit-learn	to confirm	StratifiedKFold, metrics, class weights
numpy, pandas	to confirm	Arrays, label table

Save models in the `.keras` format, not `.h5`. Under TF 2.19 the cassava weights would not reload from the older format.

Setup steps

1. Copy the project folder from Google Drive (link to be added).
2. On Windows, train inside WSL2 — native Windows TensorFlow has no GPU support after 2.10.
3. `pip install tensorflow albumentations opencv-python scikit-learn pandas`. Pin versions once confirmed.
4. Check the GPU: `python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"`.
5. Get a Kaggle API token (kaggle.com → Settings → Create New Token), put `kaggle.json` in `~/.kaggle/`, and download the competition data (see Data).
6. Run the CLAHE pre-processing cell once; it writes processed copies and a `clahe_path` column that training reads.

7. Smoke test: run one fold of the Exp 1 baseline (frozen MobileNetV2). It is fast and exercises the whole pipeline.

Kaggle

Kaggle sessions stop after a fixed time and the weekly GPU quota runs out; check your account's current limits. Save a checkpoint after every fold (fold metrics to JSON, out-of-fold predictions to `.npy`) and keep `ModelCheckpoint` on, so a timeout costs one phase rather than the run. Several training loops have an `if fold < N: continue` line for resuming — reset it before a fresh run.

Edge devices

- **Jetson Xavier NX:** inference timing only. The Pi Camera v2 (IMX219) failed on both CSI ports with I2C error -121.
- **Raspberry Pi 4:** camera capture and TensorFlow clashed over numpy builds (ABI conflict). Fix: run camera capture in a separate subprocess with its own numpy.
- **Field pipeline (CERERE):** planned chain ESP32-CAM → MQTT → FIWARE IoT Agent → Orion Context Broker → Jetson inference → Cygnus → MySQL/MongoDB. The architecture was finalised; confirm with Anas which links beyond Jetson inference were built.